KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

RETAIL SHOP INVENTORY AND PRODUCT BILLING MANAGEMENT SYSTEM

Submitted by

NAME: JEBIN

REGISTER NO: 25BCS221

Under the Guidance of

MR. DHARANEESH

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: 2025 ? 2026

APRIL 2026

Semester: II

DECLARATION

I hereby declare that the project entitled “RETAIL SHOP INVENTORY AND PRODUCT BILLING MANAGEMENT SYSTEM” submitted in partial fulfillment of the requirements for the course JAVA PROGRAMMING is a record of my work carried out under the guidance of MR. DHARANEESH, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

JEBIN

(Student)

CERTIFICATE

This is to certify that the project entitled “RETAIL SHOP INVENTORY AND PRODUCT BILLING MANAGEMENT SYSTEM” is a bona fide work carried out by JEBIN [NAME], [DEGREE / YEAR], [COMPUTER SCIENCE], KG College of Arts & Science, in partial fulfillment of the requirements for the course JAVA PROGRAMMING.

This project has been completed under my supervision and guidance during the academic period [PROJECT PERIOD].

Place: KGCAS

Date: _____

Submitted for the Viva-Voce Examination held on _____

Guide:	**Head of the Department**
(Technical Trainer, KGCAS)	(Technical Head, KGCAS)
MR. DHARANEESH	MR. BOOPALAN

TABLE OF CONTENTS

1 Introduction

1.1 Objective

1.2 Overview

1.3 Features

2 Program Analysis

2.1 Problem Definition

2.2 Project Overview

2.3 Scopes and Limitations

3 System Design

3.1 Modules

4 Methodology

4.1 Problem Analysis

4.2 System Design

4.3 Program Development

4.4 Testing and Validation

4.5 Result and Documentation

4.6 Algorithm

4.7 Flowchart

5 Source Code

6 Input and Output Screens

7 Discussion and Conclusion

7.1 Discussion

7.2 Conclusion

8 References

ABSTRACT

The Retail Shop Inventory and Product Billing Management System is a Java-based application designed to simplify product stock management and customer billing in a small retail shop. Manual inventory and billing can cause calculation mistakes, inaccurate stock counts, delays, and difficulty maintaining product details.

This project provides a simple solution by displaying available products, allowing the operator to select products and enter quantities, updating stock after a sale, and calculating the bill automatically. The bill includes product names, quantities, unit prices, line totals, subtotal, tax, and the final payable amount.

The application demonstrates Core Java concepts such as arrays, loops, conditional statements, methods, and keyboard input through a console-based interface. It is intended as an educational project and can later be extended with database storage, user authentication, and printed invoices.

The main objective is to improve billing accuracy, reduce manual work, and make basic inventory tracking easier for retail shop operators.

1. INTRODUCTION

Retail shops manage products with different prices and quantities. When stock and billing are recorded manually, the operator must look up product details, calculate item totals, add amounts, and update stock after each sale. These tasks may lead to errors, especially when several products are purchased together.

The Retail Shop Inventory and Product Billing Management System uses Java to organize product information and automate basic billing operations. The user can view products, choose products, enter quantities, and generate a bill. The program checks stock availability and reduces inventory when a sale is accepted.

1.1 Objective

• To develop a Java-based retail inventory and billing application.

• To maintain product names, prices, and available stock.

• To calculate item totals and the final bill automatically.

• To prevent sales quantities from exceeding available stock.

• To reduce manual calculation errors and save time.

1.2 Overview

The system is a menu-driven console application. It displays product details and accepts the operator’s selection. The program validates the requested quantity, calculates the cost, updates stock for completed sales, and prints a structured bill.

1.3 Features

• Product list with product ID, name, price, and stock quantity.

• Inventory display and stock availability checking.

• Multiple product selection in a single bill.

• Automatic subtotal and tax calculation.

• Stock update after each successful sale.

• Simple menu and exit option.

2. PROGRAM ANALYSIS

Program analysis identifies problems in the existing process and defines the requirements for the proposed system. It helps determine the information to store, the operations to support, and the expected output.

2.1 Problem Definition

Manual retail billing may result in incorrect calculations, outdated stock quantities, time-consuming product lookup, difficulty tracking multiple products in one sale, and inconsistent invoices. The proposed Java application addresses these problems by automating billing and stock updates.

2.2 Project Overview

• Displays the available product catalogue.

• Accepts a product ID and quantity from the user.

• Checks whether the selected quantity is available.

• Adds valid products to the current bill.

• Calculates subtotal, tax, and final amount.

• Updates remaining stock and displays the invoice.

2.3 Scopes and Limitations

Scope: Small retail shops, stationery shops, grocery shops, and educational demonstrations of billing and inventory logic.

Limitations: This version stores data in program memory only; stock changes are not saved after the program closes. It is console-based and does not include a database, barcode scanner, user accounts, or multiple-cashier support.

3. SYSTEM DESIGN

System design defines the structure of the application and the interaction between modules. The project is a menu-driven Java program with methods for displaying products, processing a sale, calculating totals, and printing the bill.

3.1 Modules

1. Product Display Module

Displays the product ID, name, unit price, and available stock.

ID	Product	Price (■)	Stock
1	Rice	50.00	10
2	Sugar	40.00	10
3	Milk	30.00	10

2. Sales Processing Module

Accepts a product ID and quantity, validates the selection, and checks that enough stock is available before adding the item to the bill.

3. Inventory Update Module

Reduces available stock by the quantity sold. The program checks quantity before confirming a sale to prevent negative stock.

4. Billing Module

Calculates each line total using Unit Price × Quantity, adds line totals to obtain the subtotal, calculates example tax, and determines the final amount.

5. Invoice and Exit Module

Prints a readable invoice and allows the operator to end the application safely.

4. METHODOLOGY

The methodology describes the stages followed to develop the project, from understanding the problem to designing, implementing, testing, and documenting the system.

• Problem Analysis

• System Design

• Program Development

• Testing and Validation

• Result and Documentation

4.1 Problem Analysis

The manual retail billing process was examined to identify tasks that could be automated.

• Manual addition may produce errors.

• Stock quantities may not be updated immediately.

• Checking product availability can delay sales.

• Handwritten bills may lack a consistent breakdown.

• Repeated calculations increase the operator’s workload.

Based on these problems, the project requirements were defined as product display, quantity validation, bill calculation, stock updating, and invoice generation.

4.2 SYSTEM DESIGN

The application is organized into small methods to make the code easier to understand and maintain.

• Product Module – stores and displays product information.

• Sales Module – receives the selected product and quantity.

• Inventory Module – checks and updates stock.

• Billing Module – calculates subtotal, tax, and total.

• Invoice Module – prints the final bill.

• Exit Module – terminates the program.

Input/output sequence:

1. Start the application.
2. Display the product menu.
3. Read the product ID and quantity.
4. Validate the product and stock availability.
5. Add the line amount to the subtotal and reduce stock.
6. Repeat until the operator finishes the sale.
7. Display the invoice and final amount.
8. End the program.

4.3 PROGRAM DEVELOPMENT

Technologies Used

• Java (JDK 8 or above).

• IDE: Eclipse, IntelliJ IDEA, or Visual Studio Code.

• Console input/output using the Scanner class.

Core Java Concepts Used

• Arrays to store product names, prices, stock, and quantities.

• Loops to display products and repeat the menu.

• Conditional statements to validate user choices.

• Methods to separate display, billing, and inventory tasks.

• Arithmetic operations for totals and tax.

Development Steps

• Created the main Java class and product data.

• Implemented product catalogue display.

• Added input handling for product IDs and quantities.

• Added stock availability validation.

• Implemented subtotal, tax, and final bill calculations.

• Created invoice output and tested the program.

4.4 TESTING AND VALIDATION

Testing checks whether the application displays product information correctly, accepts valid input, calculates the bill accurately, and prevents sales beyond available stock.

Testing Done

• Verified product names, prices, and stock values.

• Checked valid product selection and quantity input.

• Checked multiple products in one bill.

• Verified subtotal, tax, and final amount.

• Tested insufficient stock and invalid product IDs.

• Checked the finish-order option.

Test Case	Input / Condition	Expected Result
Valid sale	Product 1, qty 2	Subtotal ■100; stock reduced by 2
Multiple items	Rice and Milk	Both items appear on invoice
Insufficient stock	Request above available stock	Sale rejected; stock unchanged
Invalid product	ID 9	Error message displayed
Finish order	ID 0	Invoice generated

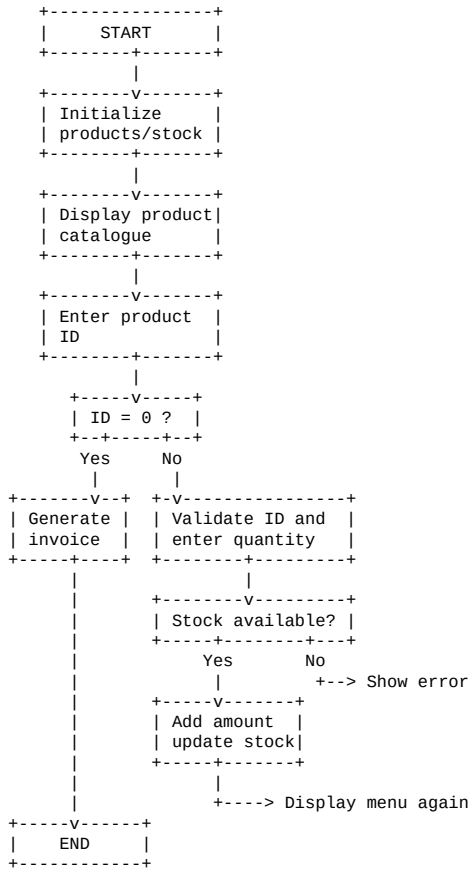
4.5 RESULT AND DOCUMENTATION

After implementing and testing the application, it was able to display product details, process valid sales, calculate bills, update stock, and generate invoices. The documentation explains project objectives, requirements, modules, methodology, algorithm, source code, sample output, and conclusion.

4.6 ALGORITHM

1. Start the program.
2. Initialize product names, prices, and stock quantities.
3. Display the available product list.
4. Ask the operator to enter a product ID.
5. If the ID is 0, finish the sale and generate the invoice.
6. Otherwise, validate the product ID.
7. Ask for quantity and check that it is positive and available.
8. If valid, calculate price × quantity, add it to subtotal, and reduce stock.
9. If invalid or stock is insufficient, display an error message.
10. Repeat for other products.
11. Calculate tax and final amount.
12. Display the invoice and end the program.

4.7 FLOWCHART



5. SOURCE CODE

This Core Java program provides a console-based product inventory and billing workflow. The sample catalogue contains Rice, Sugar, and Milk. The 5% tax is only a demonstration value and should be adjusted to the shop's applicable requirements.

```
import java.util.Scanner;

public class RetailShopSystem {
    static Scanner sc = new Scanner(System.in);
    static String[] products = {"Rice", "Sugar", "Milk"};
    static double[] prices = {50.0, 40.0, 30.0};
    static int[] stock = {10, 10, 10};
    static int[] soldQty = new int[products.length];
    static double subtotal = 0.0;

    public static void main(String[] args) {
        System.out.println("===== RETAIL SHOP BILLING =====");
        int choice;
        do {
            displayProducts();
            System.out.print("Enter product ID (0 to finish): ");
            choice = sc.nextInt();
            if (choice == 0) break;
            if (choice < 1 || choice > products.length) {
                System.out.println("Invalid product ID!");
                continue;
            }
            System.out.print("Enter quantity: ");
            int qty = sc.nextInt();
            if (qty <= 0) {
                System.out.println("Quantity must be greater than zero.");
            } else if (qty > stock[choice - 1]) {
                System.out.println("Insufficient stock available!");
            } else {
                int i = choice - 1;
                soldQty[i] += qty;
                subtotal += prices[i] * qty;
                stock[i] -= qty;
                System.out.println("Item added to bill.");
            }
        } while (true);
        generateBill();
        displayStock();
        sc.close();
    }
```

5. SOURCE CODE (CONTINUED)

```
    static void displayProducts() {
        System.out.println("\\nAvailable Products:");
        System.out.printf("%-5s %-12s %-10s %-8s%n",
                          "ID", "Product", "Price", "Stock");
        for (int i = 0; i < products.length; i++) {
            System.out.printf("%-5d %-12s Rs.%-7.2f %-8d%n",
                    i + 1, products[i], prices[i], stock[i]);
        }
    }

    static void generateBill() {
        double tax = subtotal * 0.05;
        double grandTotal = subtotal + tax;
        System.out.println("\\n========== CUSTOMER BILL ==========");
        System.out.printf("%-12s %-8s %-10s%n",
                          "Product", "Qty", "Amount");
        for (int i = 0; i < products.length; i++) {
            if (soldQty[i] > 0) {
                double amount = soldQty[i] * prices[i];
                System.out.printf("%-12s %-8d Rs.%-7.2f%n",
                        products[i], soldQty[i], amount);
            }
        }
        System.out.println("-----------------------------------");
        System.out.printf("Subtotal:       Rs.%.2f%n", subtotal);
        System.out.printf("Tax (5%%):       Rs.%.2f%n", tax);
        System.out.printf("Grand Total:    Rs.%.2f%n", grandTotal);
        System.out.println("Thank you for shopping!");
    }

    static void displayStock() {
        System.out.println("\\nRemaining Stock:");
        for (int i = 0; i < products.length; i++) {
            System.out.println(products[i] + ": " + stock[i]);
        }
    }
}
```

6. INPUT AND OUTPUT SCREENS

Sample Console Input and Output

```
===== RETAIL SHOP BILLING =====

Available Products:
ID    Product      Price      Stock
1     Rice         Rs.50.00   10
2     Sugar        Rs.40.00   10
3     Milk         Rs.30.00   10
Enter product ID (0 to finish): 1
Enter quantity: 2
Item added to bill.

Enter product ID (0 to finish): 3
Enter quantity: 1
Item added to bill.

Enter product ID (0 to finish): 0

========== CUSTOMER BILL ==========
Product      Qty      Amount
Rice         2        Rs.100.00
Milk         1        Rs.30.00
-----------------------------------
Subtotal:       Rs.130.00
Tax (5%):       Rs.6.50
Grand Total:    Rs.136.50
Thank you for shopping!

Remaining Stock:
Rice: 8
Sugar: 10
Milk: 9
```

The example demonstrates a sale of two units of Rice and one unit of Milk. The subtotal is ■130.00, example tax is ■6.50, and the final amount is ■136.50. Stock quantities are reduced after the items are accepted.

7. DISCUSSION AND CONCLUSION

7.1 Discussion

The Retail Shop Inventory and Product Billing Management System was developed to demonstrate how Java can automate basic retail shop operations. The application allows the operator to view products, enter quantities, check stock availability, calculate the bill, and display remaining inventory.

Core Java concepts such as arrays, loops, conditional statements, methods, formatted output, and the Scanner class are used in the implementation. Testing with valid and invalid inputs helps confirm that the system calculates totals correctly and does not allow a sale when the requested quantity exceeds available stock.

The current version keeps data in memory, so inventory changes are lost when the program closes. Future improvements could include database connectivity, permanent sales records, product addition and editing, barcode scanning, user login, and a graphical user interface.

7.2 Conclusion

The project demonstrates a simple Java-based solution for retail product inventory and billing. It reduces manual calculations, checks product availability, updates stock after a sale, and generates a clear invoice with subtotal, tax, and final payable amount.

The system meets the basic educational objectives and provides a foundation for a more advanced retail management application.

8. REFERENCES

Books

• Herbert Schildt – Java: The Complete Reference.

• Kathy Sierra and Bert Bates – Head First Java.

Websites

• Oracle Java Documentation: https://docs.oracle.com/javase/8/docs/

• GeeksforGeeks Java: https://www.geeksforgeeks.org/java/

• W3Schools Java Tutorial: https://www.w3schools.com/java/

• Javatpoint Java Tutorial: https://www.javatpoint.com/java-tutorial